
Batch 11 — Admin Operational Forms Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:13

Batch 11 — Admin Operational Forms Pack

Meat Lovers CIMS powered by YohPal

11A. Update

apps/admin-web/src/features/operations/api/operationsApi.js

Add these methods inside operationsApi:

```
createSupplier: async (payload) => {
  const { data } = await apiClient.post('/suppliers', payload)
  return data
},

createProduct: async (payload) => {
  const { data } = await apiClient.post('/products', payload)
  return data
},

stockIn: async (payload) => {
  const { data } = await apiClient.post('/inventory/stock-in', payload)
  return data
},

barStockIssue: async (payload) => {
  const { data } = await apiClient.post('/bar/stock-issue', payload)
  return data
},

markKitchenReady: async (orderId) => {
  const { data } = await apiClient.post(`/kitchen/orders/${orderId}/mark-ready`, {})
  return data
},

dispatchOrder: async (orderId, payload) => {
  const { data } = await apiClient.post(`/dispatch/orders/${orderId}/dispatch`, payload)
  return data
},

markDelivered: async (orderId) => {
  const { data } = await apiClient.post(`/dispatch/orders/${orderId}/deliver`, {})
  return data
},

declareUnsoldFood: async (payload) => {
  const { data } = await apiClient.post('/unsold-food', payload)
  return data
},

createAsset: async (payload) => {
  const { data } = await apiClient.post('/assets', payload)
  return data
},

createStaff: async (payload) => {
  const { data } = await apiClient.post('/hrm/staff', payload)
  return data
},

createFinanceTransaction: async (payload) => {
  const { data } = await apiClient.post('/finance/transactions', payload)
  return data
},
```

11B.

apps/admin-web/src/components/common/OperationalForm.jsx

```
import React from 'react'
import Button from '../ui/Button'
import Input from '../ui/Input'
import Select from '../ui/Select'

export default function OperationalForm({ fields, form, setForm, submitLabel = 'Save', onSubmit }) {
  return (
    <form onSubmit={onSubmit} style={{ display: 'grid', gap: 10, maxWidth: 560 }}>
      {fields.map((field) => {
        if (field.type === 'select') {
          return (
            <Select
              key={field.name}
              value={form[field.name] ?? ''}
              onChange={(e) => setForm({ ...form, [field.name]: e.target.value })}
            >
              <option value="">{field.placeholder || `Select ${field.label}`}</option>
              {field.options.map((option) => (
                <option key={option.value} value={option.value}>
                  {option.label}
                </option>
              ))}
            </Select>
          )
        }

        if (field.type === 'textarea') {
          return (
            <textarea
              key={field.name}
              placeholder={field.placeholder || field.label}
              value={form[field.name] ?? ''}
              onChange={(e) => setForm({ ...form, [field.name]: e.target.value })}
              style={{
                padding: '10px 12px',
                border: '1px solid #d1d5db',
                borderRadius: 10,
                width: '100%',
                minHeight: 90,
              }}
            />
          )
        }

        return (
          <Input
            key={field.name}
            type={field.type || 'text'}
            placeholder={field.placeholder || field.label}
            value={form[field.name] ?? ''}
            onChange={(e) => setForm({ ...form, [field.name]: e.target.value })}
          />
        )
      })}

      <Button type="submit">{submitLabel}</Button>
    </form>
  )
}
```

11C.

apps/admin-web/src/components/common/OperationalFormPage.jsx

```
import React, { useState } from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../ui/Page'
import Card from '../ui/Card'
import DataTable from '../ui/DataTable'
import LoadingBlock from '../ui/LoadingBlock'
import StatusMessage from '../ui/StatusMessage'
import OperationalForm from './OperationalForm'
import { safeMessage } from '../../lib/helpers'

export default function OperationalFormPage({
  title,
  subtitle,
  queryKey,
  queryFn,
  mutationFn,
  initialForm,
  buildPayload,
}) {
  const [form, setForm] = useState(initialForm)
  const [message, setMessage] = useState('')
  const [status, setStatus] = useState('')

  const query = useQuery({
    queryKey,
    queryFn,
  })

  const mutation = useMutation({
    mutationFn,
    onSuccess: () => {
      queryClient.setQueryData(queryKey, queryFn())
      setMessage(safeMessage('Success'))
      setStatus('success')
    },
    onError: (error) => {
      setMessage(safeMessage('Error'))
      setStatus('error')
    },
  })

  return (
    <Page>
      <Card>
        <h3>{title}</h3>
        <p>{subtitle}</p>
        <OperationalForm
          fields={fields}
          form={form}
          setForm={setForm}
          submitLabel={submitLabel}
          onSubmit={onSubmit}
        />
      </Card>
      <StatusMessage status={status} message={message} />
    </Page>
  )
}
```

```

    fields,
    columns,
    mapRows,
    submitLabel = 'Save',
  }) {
    const qc = useQueryClient()
    const [form, setForm] = useState(initialForm)
    const [success, setSuccess] = useState('')

    const { data, isLoading, error } = useQuery({
      queryKey: [queryKey],
      queryFn,
    })

    const mutation = useMutation({
      mutationFn,
      onSuccess: () => {
        setSuccess('Saved successfully')
        setForm(initialForm)
        qc.invalidateQueries({ queryKey: [queryKey] })
      },
    })

    function submit(e) {
      e.preventDefault()
      setSuccess('')
      mutation.mutate(buildPayload(form))
    }

    const rows = mapRows ? mapRows(data) : data || []

    return (
      <Page title={title} subtitle={subtitle}>
        <Card title="Create / Record">
          <StatusMessage success={success} error={safeMessage(mutation.error, '')} />
          <OperationalForm
            fields={fields}
            form={form}
            setForm={setForm}
            submitLabel={submitLabel}
            onSubmit={submit}
          />
        </Card>

        <Card title="Records">
          {isLoading ? <LoadingBlock label={`Loading ${title.toLowerCase()}...`} /> : null}
          {error ? <StatusMessage error={safeMessage(error, 'Could not load records')} /> : null}
          {isLoading && !error ? <DataTable columns={columns} rows={rows} /> : null}
        </Card>
      </Page>
    )
  }
}

```

11D. Replace

apps/admin-web/src/features/operations/pages/SuppliersOperationalPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../api/operationsApi'

export default function SuppliersOperationalPage() {
  return (
    <OperationalFormPage
      title="Suppliers"
      subtitle="Create and monitor supplier records"
      queryKey="ops-suppliers"
      queryFn={operationsApi.suppliers}
      mutationFn={operationsApi.createSupplier}
      initialForm={{
        supplier_name: '',
        contact_person: '',
        phone: '',
        email: '',
        physical_address: '',
        supplier_type: 'FOOD',
        status: 'ACTIVE',
      }}
      buildPayload={({form}) => form}
      fields={[
        { name: 'supplier_name', label: 'Supplier Name' },
        { name: 'contact_person', label: 'Contact Person' },
        { name: 'phone', label: 'Phone' },
        { name: 'email', label: 'Email' },
      ]}
    />
  )
}

```

```

    { name: 'physical_address', label: 'Physical Address', type: 'textarea' },
    {
      name: 'supplier_type',
      label: 'Supplier Type',
      type: 'select',
      options: [
        { value: 'FOOD', label: 'Food' },
        { value: 'SOFT_DRINKS', label: 'Soft Drinks' },
        { value: 'ALCOHOL', label: 'Alcohol' },
        { value: 'GENERAL', label: 'General' },
      ],
    },
  ],
  {
    name: 'status',
    label: 'Status',
    type: 'select',
    options: [
      { value: 'ACTIVE', label: 'Active' },
      { value: 'SUSPENDED', label: 'Suspended' },
    ],
  },
],
})
}
)
/>
}

```

11E. Replace

apps/admin-web/src/features/operations/pages/ProductsOperationalPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../api/operationsApi'

export default function ProductsOperationalPage() {
  return (
    <OperationalFormPage
      title="Products"
      subtitle="Create food, soft drink, and alcoholic drink products"
      queryKey="ops-products"
      queryFn={operationsApi.products}
      mutationFn={operationsApi.createProduct}
      initialForm={{
        product_name: '',
        product_category: 'FOOD',
        selling_price: '',
        cost_price: '',
        barcode: '',
      }}
      buildPayload={({form}) => ({
        ...form,
        selling_price: Number(form.selling_price),
        cost_price: Number(form.cost_price),
      })
    >
      fields=[
        { name: 'product_name', label: 'Product Name' },
        {
          name: 'product_category',
          label: 'Product Category',
          type: 'select',
          options: [
            { value: 'FOOD', label: 'Food' },
            { value: 'SOFT_DRINK', label: 'Soft Drink' },
            { value: 'ALCOHOLIC_DRINK', label: 'Alcoholic Drink' },
          ],
        },
        { name: 'selling_price', label: 'Selling Price', type: 'number' },
        { name: 'cost_price', label: 'Cost Price', type: 'number' },
        { name: 'barcode', label: 'Barcode Optional' },
      ],
      columns=[
        { key: 'product_name', label: 'Product' },
        { key: 'product_category', label: 'Category' },
        { key: 'selling_price', label: 'Selling Price' },
        { key: 'cost_price', label: 'Cost Price' },
        { key: 'is_active', label: 'Active' },
      ],
    </OperationalFormPage>
  )
}

```

```

    )
  }
}

```

11F. Replace

apps/admin-web/src/features/operations/pages/StockOperationalPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../api/operationsApi'

export default function StockOperationalPage() {
  return (
    <OperationalFormPage
      title="Stock"
      subtitle="Record stock-in and monitor storekeeping balance"
      queryKey="ops-stock"
      queryFn={operationsApi.stock}
      mutationFn={operationsApi.stockIn}
      initialForm={{
        product_id: '',
        movement_type: 'PURCHASE',
        quantity: '',
        reference_number: '',
        notes: '',
      }}
      buildPayload={({form}) => ({
        ...form,
        product_id: Number(form.product_id),
        quantity: Number(form.quantity),
      })
    }
      fields={[
        { name: 'product_id', label: 'Product ID', type: 'number' },
        {
          name: 'movement_type',
          label: 'Movement Type',
          type: 'select',
          options: [
            { value: 'PURCHASE', label: 'Purchase' },
            { value: 'STORE_TO_KITCHEN', label: 'Store to Kitchen' },
            { value: 'STORE_TO_BAR', label: 'Store to Bar' },
            { value: 'ADJUSTMENT', label: 'Adjustment' },
          ],
        },
        { name: 'quantity', label: 'Quantity', type: 'number' },
        { name: 'reference_number', label: 'Reference Number' },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'product_name', label: 'Product' },
        { key: 'product_category', label: 'Category' },
        { key: 'current_quantity', label: 'Current Quantity' },
        { key: 'reorder_level', label: 'Reorder Level' },
      ]}
    />
  )
}

```

11G. Replace

apps/admin-web/src/features/operations/pages/BarOperationalPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../api/operationsApi'

export default function BarOperationalPage() {
  return (
    <OperationalFormPage
      title="Bar"
      subtitle="Issue bar stock and monitor alcoholic drinks accountability"
      queryKey="ops-bar"
      queryFn={operationsApi.barStock}
      mutationFn={operationsApi.barStockIssue}
      initialForm={{
        product_id: '',
        quantity: '',
        reference_number: '',
        notes: '',
      }}
      buildPayload={({form}) => ({

```

```

    ...form,
    product_id: Number(form.product_id),
    quantity: Number(form.quantity),
  })}
  fields=[
    { name: 'product_id', label: 'Alcoholic Drink Product ID', type: 'number' },
    { name: 'quantity', label: 'Quantity Issued', type: 'number' },
    { name: 'reference_number', label: 'Reference Number' },
    { name: 'notes', label: 'Notes', type: 'textarea' },
  ]
  columns=[
    { key: 'product_name', label: 'Drink' },
    { key: 'product_category', label: 'Category' },
    { key: 'current_quantity', label: 'Current Quantity' },
    { key: 'reorder_level', label: 'Reorder Level' },
  ]
  submitLabel="Issue Bar Stock"
/>
)
}

```

11H. Replace

apps/admin-web/src/features/operations/pages/KitchenOperationalPage.jsx

```

import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../api/operationsApi'

export default function KitchenOperationalPage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['ops-kitchen'],
    queryFn: operationsApi.kitchenOrders,
  })

  const mutation = useMutation({
    mutationFn: operationsApi.markKitchenReady,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['ops-kitchen'] }),
  })

  const rows = (data || []).map((order) => ({
    ...order,
    action: <Button onClick={() => mutation.mutate(order.id)}>Mark Ready</Button>,
  })))

  return (
    <Page title="Kitchen" subtitle="Food production queue and kitchen readiness">
      {mutation.error ? <StatusMessage error={mutation.error.message} /> : null}

      <Card>
        {isLoading ? <LoadingBlock label="Loading kitchen orders..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns=[
              { key: 'order_number', label: 'Order No.' },
              { key: 'table_number', label: 'Table' },
              { key: 'order_status', label: 'Status' },
              { key: 'created_at', label: 'Created' },
              { key: 'action', label: 'Action' },
            ]
            rows={rows}
          />
        ) : null}
      </Card>
    </Page>
  )
}

```

11I. Replace

apps/admin-web/src/features/operations/pages/DispatchOperationalPage.jsx

```

import React, { useState } from 'react'

```

```

import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../components/ui/Page'
import Card from '../../components/ui/Card'
import Button from '../../components/ui/Button'
import Input from '../../components/ui/Input'
import DataTable from '../../components/ui/DataTable'
import LoadingBlock from '../../components/ui/LoadingBlock'
import StatusMessage from '../../components/ui/StatusMessage'
import { operationsApi } from '../api/operationsApi'

export default function DispatchOperationalPage() {
  const qc = useQueryClient()
  const [dispatchForm, setDispatchForm] = useState({
    order_id: '',
    rider_name: '',
    rider_phone: '',
  })

  const { data, isLoading, error } = useQuery({
    queryKey: ['ops-dispatch'],
    queryFn: operationsApi.dispatchOrders,
  })

  const dispatchMutation = useMutation({
    mutationFn: (payload) =>
      operationsApi.dispatchOrder(payload.order_id, {
        rider_name: payload.rider_name,
        rider_phone: payload.rider_phone,
      }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['ops-dispatch'] }),
  })

  const deliveredMutation = useMutation({
    mutationFn: operationsApi.markDelivered,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['ops-dispatch'] }),
  })

  const rows = (data || []).map((item) => ({
    ...item,
    action: <Button onClick={() => deliveredMutation.mutate(item.order_id)}>Mark Delivered</Button>,
  }))

  return (
    <Page title="Dispatch" subtitle="Delivery dispatch queue and rider tracking">
      <Card title="Dispatch Order">
        <form
          onSubmit={(e) => {
            e.preventDefault()
            dispatchMutation.mutate(dispatchForm)
          }}
          style={{ display: 'grid', gap: 10, maxWidth: 560 }}
        >
          <Input
            placeholder="Order ID"
            value={dispatchForm.order_id}
            onChange={(e) => setDispatchForm({ ...dispatchForm, order_id: e.target.value })}
          />
          <Input
            placeholder="Rider Name"
            value={dispatchForm.rider_name}
            onChange={(e) => setDispatchForm({ ...dispatchForm, rider_name: e.target.value })}
          />
          <Input
            placeholder="Rider Phone"
            value={dispatchForm.rider_phone}
            onChange={(e) => setDispatchForm({ ...dispatchForm, rider_phone: e.target.value })}
          />
          <Button type="submit">Dispatch Order</Button>
        </form>
      </Card>

      <Card title="Deliveries">
        {isLoading ? <LoadingBlock label="Loading deliveries..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {dispatchMutation.error ? <StatusMessage error={dispatchMutation.error.message} /> : null}
        {deliveredMutation.error ? <StatusMessage error={deliveredMutation.error.message} /> : null}

        {(!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'order_number', label: 'Order No.' },
              { key: 'customer_name', label: 'Customer' },
              { key: 'delivery_status', label: 'Status' },
              { key: 'rider_name', label: 'Rider' },
              { key: 'rider_phone', label: 'Rider Phone' },
              { key: 'action', label: 'Action' },
            ]}
          />
        ) : null)}
      </Card>
    </Page>
  )
}

```

```

        rows={rows}
      />
    ) : null}
  </Card>
</Page>
)
}

```

11J. Replace

apps/admin-web/src/features/operations/pages/UnsoldFoodOperationalPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../api/operationsApi'

export default function UnsoldFoodOperationalPage() {
  return (
    <OperationalFormPage
      title="Unsold Cooked Food"
      subtitle="Declare unsold food for waste and theft control"
      queryKey="ops-unsold-food"
      queryFn={operationsApi.unsoldFood}
      mutationFn={operationsApi.declareUnsoldFood}
      initialForm={{
        product_id: '',
        quantity: '',
        reason: '',
      }}
      buildPayload={(form) => ({
        ...form,
        product_id: Number(form.product_id),
        quantity: Number(form.quantity),
      })}
      fields={[
        { name: 'product_id', label: 'Food Product ID', type: 'number' },
        { name: 'quantity', label: 'Quantity', type: 'number' },
        { name: 'reason', label: 'Reason', type: 'textarea' },
      ]}
      columns={[
        { key: 'product_name', label: 'Product' },
        { key: 'quantity', label: 'Quantity' },
        { key: 'reason', label: 'Reason' },
        { key: 'declared_by_name', label: 'Declared By' },
        { key: 'created_at', label: 'Date' },
      ]}
      submitLabel="Declare Unsold Food"
    />
  )
}

```

11K. Replace

apps/admin-web/src/features/operations/pages/AssetsOperationalPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../api/operationsApi'

export default function AssetsOperationalPage() {
  return (
    <OperationalFormPage
      title="Assets"
      subtitle="Create and monitor restaurant asset inventory"
      queryKey="ops-assets"
      queryFn={operationsApi.assets}
      mutationFn={operationsApi.createAsset}
      initialForm={{
        asset_name: '',
        asset_category: '',
        serial_number: '',
        purchase_cost: '',
        asset_status: 'ACTIVE',
      }}
      buildPayload={(form) => ({
        ...form,
        purchase_cost: Number(form.purchase_cost),
      })}
      fields={[
        { name: 'asset_name', label: 'Asset Name' },
        { name: 'asset_category', label: 'Asset Category' },
        { name: 'serial_number', label: 'Serial Number' },
      ]}
    />
  )
}

```

```

    { name: 'purchase_cost', label: 'Purchase Cost', type: 'number' },
    {
      name: 'asset_status',
      label: 'Asset Status',
      type: 'select',
      options: [
        { value: 'ACTIVE', label: 'Active' },
        { value: 'DAMAGED', label: 'Damaged' },
        { value: 'DISPOSED', label: 'Disposed' },
      ],
    },
  ],
}
})
}
columns=[
  { key: 'asset_name', label: 'Asset' },
  { key: 'asset_category', label: 'Category' },
  { key: 'serial_number', label: 'Serial No.' },
  { key: 'purchase_cost', label: 'Cost' },
  { key: 'asset_status', label: 'Status' },
]
})
/>
)
}

```

11L. Replace

apps/admin-web/src/features/operations/pages/HRMOperationalPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../api/operationsApi'

export default function HRMOperationalPage() {
  return (
    <OperationalFormPage
      title="HRM"
      subtitle="Create staff users and manage role visibility"
      queryKey="ops-hrm"
      queryFn={operationsApi.staff}
      mutationFn={operationsApi.createStaff}
      initialForm={{
        full_name: '',
        email: '',
        phone: '',
        password: '',
        role: 'WAITER',
      }}
      buildPayload={(form) => form}
      fields=[
        { name: 'full_name', label: 'Full Name' },
        { name: 'email', label: 'Email' },
        { name: 'phone', label: 'Phone' },
        { name: 'password', label: 'Temporary Password', type: 'password' },
        {
          name: 'role',
          label: 'Role',
          type: 'select',
          options: [
            { value: 'SUPER_ADMIN', label: 'Super Admin' },
            { value: 'ADMIN', label: 'Admin' },
            { value: 'MANAGER', label: 'Manager' },
            { value: 'CASHIER', label: 'Cashier' },
            { value: 'WAITER', label: 'Waiter' },
            { value: 'CHEF', label: 'Chef' },
            { value: 'STOREKEEPER', label: 'Storekeeper' },
            { value: 'BARMAN', label: 'Barman' },
            { value: 'DISPATCHER', label: 'Dispatcher' },
            { value: 'ACCOUNTANT', label: 'Accountant' },
            { value: 'HR', label: 'HR' },
          ],
        },
      ],
    >
    </OperationalFormPage>
  )
}

```

11M. Replace

apps/admin-web/src/features/operations/pages/FinanceOperationalPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../api/operationsApi'

export default function FinanceOperationalPage() {
  return (
    <OperationalFormPage
      title="Finance"
      subtitle="Record income and expense transactions"
      queryKey="ops-finance"
      queryFn={operationsApi.financeTransactions}
      mutationFn={operationsApi.createFinanceTransaction}
      initialForm={{
        transaction_type: 'EXPENSE',
        category: '',
        amount: '',
        reference_number: '',
        notes: '',
      }}
      buildPayload={(form) => ({
        ...form,
        amount: Number(form.amount),
      })}
      fields={[
        {
          name: 'transaction_type',
          label: 'Transaction Type',
          type: 'select',
          options: [
            { value: 'INCOME', label: 'Income' },
            { value: 'EXPENSE', label: 'Expense' },
          ],
        },
        { name: 'category', label: 'Category' },
        { name: 'amount', label: 'Amount', type: 'number' },
        { name: 'reference_number', label: 'Reference Number' },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'transaction_type', label: 'Type' },
        { key: 'category', label: 'Category' },
        { key: 'amount', label: 'Amount' },
        { key: 'reference_number', label: 'Reference' },
        { key: 'created_by_name', label: 'Created By' },
        { key: 'created_at', label: 'Date' },
      ]}
      submitLabel="Record Transaction"
    />
  )
}
```

11N.

docs/ADMIN_OPERATIONAL_FORMS_PACK.md

Batch 11 – Admin Operational Forms Pack

System

Meat Lovers CIMS powered by YohPal

Purpose

This batch turns operational admin pages from read-only visibility into action-ready screens.

Forms Added

1. Supplier creation
2. Product creation
3. Stock-in
4. Bar stock issue
5. Kitchen mark ready action
6. Dispatch action
7. Delivery mark delivered action
8. Unsold food declaration
9. Asset creation
10. Staff creation
11. Finance transaction creation

```
## Business Control Value

### Supplier Creation
Management can register approved suppliers.

### Product Creation
Products are categorized into:
- food
- soft drinks
- alcoholic drinks

### Stock-In
Storekeeping can record incoming stock.

### Bar Stock Issue
Bar movement is separated from food stock.

### Kitchen Mark Ready
Kitchen can update order readiness.

### Dispatch Action
Delivery movement can be recorded.

### Unsold Food Declaration
Unsold cooked food is tracked for waste and theft control.

### Asset Creation
Equipment and business assets can be registered.

### Staff Creation
HRM can create role-based staff records.

### Finance Transaction Creation
Income and expenses can be recorded for P&L.

## Developer Note

This batch introduces:

- `OperationalForm`
- `OperationalFormPage`

These reusable components should be used for future admin forms.
```

Batch 11 Outcome

Admin operational screens now support:

- ✓ supplier creation
- ✓ product creation
- ✓ stock-in
- ✓ bar stock issue
- ✓ kitchen mark ready action
- ✓ dispatch action
- ✓ unsold food declaration
- ✓ asset creation
- ✓ staff creation
- ✓ finance transaction creation

Next Smart Move

Build **Batch 12 — POS PWA Waiter Smartphone App**, including:

- waiter login
- mobile menu
- table selection
- cart
- create order
- order status
- cash/M-Pesa pending payment state
- cancel request
- discount request
- mobile-first POS navigation